

A project report on
**SENTRYSCOPE: LOW-POWER
PERIMETER MONITORING**

PROJECT REPORT

Submitted for CBCS in B.Tech – ECE
BECE320E – Embedded C Programming

by

Gopikasree R – 23BEC1013
Kirthana S – 23BEC1412
Harshitha Senthil Kumar – 23BEC1428
Joshitha G – 23BEC1478

Slot: B1

Name of faculty: Dr. A. Sivasubramanian



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF ELECTRONICS ENGINEERING

April 2026

DECLARATION

I hereby declare that the thesis entitled “SENTRYSCOPE: LOW-POWER PERIMETER MONITORING” submitted by Gopikasree R (23BEC1013), Kirthana S (23BEC1412), Harshitha Senthil Kumar (23BEC1428), Joshitha G(23BEC1478) for the award of the degree of Bachelor of Technology in Electronics and Communication , Vellore Institute of Technology, Chennai is a record of Bonafide work carried out by me under the supervision of Dr. A. Sivasubramanian

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: 1- 04 0 2026

Gopikasree R (23BEC1013)

Kirthana S (23BEC1412)

Harshitha Senthil Kumar (23BEC1428)

Joshitha G (23BEC1478)

TABLE OF CONTENTS

Description	Page No
ABSTRACT	6
LIST OF TABLES	7
LIST OF FIGURES	8
LIST OF SYMBOLS AND ABBREVIATIONS	9
CHAPTER- 1 INTRODUCTION	
1.1.Need/ Motivation for the Project	10
1.2.Innovative Content in the Project	11
1.3.Block Schematic	12
1.3.1 Module descriptions	12
1.3.2 Cost of the Project	15
1.3.3 Advantages and Disadvantages	16
1.4.Overview of the Report	17
CHAPTER - 2 LITERATURE SURVEY	
2.1. Survey On IOT-Based Intrusion Detection Systems	18
2.2 Survey On ESP32-Based Smart Surveillance Systems	18
2.3 Survey on Multi-Sensor Intrusion Detection Approaches	19
2.4 Survey on Ultrasonic-Based Detection Systems	19
2.5 Survey on Camera-Based Intrusion Detection Systems	20
2.6 Survey on Actuation and Control Mechanisms	20
2.7 Research Gap and Motivation	21
CHAPTER - 3 DESIGN AND SIMULATION	
3.1 Design of the Project	22
3.1.1. System design Architecture	22
3.1.2. Working Principle of the System	24

3.1.3. Software design and Control Strategy	25
3.2 Simulation	26
3.2.1 ESP32 Control logic systems	26
3.2.2 ESP32 Cam Livestream Simulation	26
3.2.3 Integrating System behaviours	28
3.2.4 Codes	28
3.3 Screenshots of the simulated results	35
CHAPTER - 4 HARDWARE	
4.1 Hardware Description	37
4.1.1 Sensing and Processing	37
4.1.2 Actuation, Imaging & Supporting Unit	37
4.1.3 Hardware Specifications	38
4.2 Results and Discussion	39
4.3 Screenshots of the Hardware used and Results	41
CHAPTER -5 CONCLUSION AND FUTURE SCOPE	
5.1 Conclusions	42
5.2 Future Scope	43
BIBLIOGRAPHY	44

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. A. Sivasubramanian, Professor, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Embedded C.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Dr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G.V. Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai, Dr. K. Sathiyarayanan, Director, Chennai Campus and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

In jubilant state, I express ingeniously my whole-hearted thanks to Dr. M. Jagannath, Head of the Department, B.Tech. Electronics and Communication and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staff at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: 1-4-2026

Gopikasree R, Kirthana S, Harshitha Senthil Kumar, Joshitha G

ABSTRACT

The increasing need for intelligent and energy-efficient surveillance systems has driven the development of embedded solutions capable of real-time monitoring and automated response. This project presents the design and implementation of a multi-sensor-based intrusion detection and smart camera system using the ESP32 microcontroller platform.

The proposed system integrates three distinct sensing mechanisms: an infrared (IR) proximity sensor for detecting nearby motion, an IR break-beam sensor for identifying line-of-sight interruptions, and an ultrasonic sensor for distance-based detection. By combining these sensing techniques, the system significantly reduces false positives commonly associated with single-sensor systems and improves overall detection reliability.

Upon detecting an intrusion, the ESP32 controller processes sensor inputs and determines the nature and location of the event. A servo motor is then actuated to orient the camera toward the detected region, ensuring targeted monitoring. Subsequently, an ESP32-CAM module is triggered to capture an image of the intrusion event, which is stored locally on a microSD card for later analysis. The system employs a finite-state machine architecture to manage operations such as detection, camera triggering, servo movement, and system reset, ensuring smooth and non-overlapping execution.

The system is cost-effective, modular, and suitable for deployment in residential, industrial, and agricultural environments. It also provides a strong foundation for future integration with IoT platforms and AI-based analytics, making it a scalable solution for next-generation smart surveillance systems.

LIST OF FIGURES

S.NO	FIGURE DESCRIPTION	Page No
1	Overall System schematic	12
2	Hardware Block Diagram	22
3	Complete Hardware Setup	23
4	System Principle	24
5	Software Architecture	26
6	IDE Environment Results	35
7	Camera Livestream	36
8	SD Card Storage	36
9	Detection of IR proximity and Ultrasonic Sensors	41
10	Detection of IR Break Beam Sensor	41

LIST OF TABLES

TABLE DESCRIPTION	Page No
Table 1: Cost of the Project	15
Table 2: Hardware Specifications of System Components	38

LIST OF SYMBOLS AND ABBREVIATIONS

1. SYMBOLS

- d – Distance measured by ultrasonic sensor
- t – Time delay of ultrasonic signal
- θ – Servo motor rotation angle
- P – Power consumption of the system
- V – Voltage level (e.g., 5V, 3.3V supply)

2. ABBREVIATIONS

- ESP32 – Espressif 32-bit Microcontroller
- IR – Infrared
- GPIO – General Purpose Input Output
- PWM – Pulse Width Modulation
- FSM – Finite State Machine
- SD – Secure Digital (memory card)
- IDE – Integrated Development Environment
- IoT – Internet of Things
- Wi-Fi – Wireless Fidelity
- LED – Light Emitting Diode
- HTTP – HyperText Transfer Protocol
- MCU – Microcontroller Unit

CHAPTER – 1

INTRODUCTION

1.1 MOTIVATION FOR THE PROJECT

The primary motivation behind this project is to develop an energy-efficient and intelligent surveillance system that operates only, when necessary, rather than relying on continuous monitoring. Traditional security systems often record footage around the clock, leading to significant wastage of power, storage resources, and processing capacity. In contrast, this system is designed to activate the camera only during verified intrusion events. By triggering image capture selectively, the system minimizes unnecessary operation and optimizes both energy consumption and data storage, making it more suitable for long-term and large-scale deployments, particularly in modern Internet of Things (IoT) based environments.

Another key motivation is to improve the reliability and trustworthiness of intrusion detection. Systems that depend on a single sensing mechanism are highly prone to false alarms caused by environmental noise, lighting variations, or unintended disturbances. To address this limitation, the proposed design incorporates multiple independent sensing modes, including Infrared (IR) proximity detection, break-beam interruption, and ultrasonic distance measurement. By combining these complementary sensing techniques, the system ensures that detection is validated through multiple inputs, significantly reducing nuisance alerts and enhancing overall accuracy. This multi-sensor fusion approach increases user confidence in the system and ensures that alerts correspond to genuine events.

Furthermore, there is a growing demand for surveillance systems that are practical, adaptable, and easy to deploy across different environments. Whether in residential areas, industrial facilities, or remote locations, users require solutions that are simple to install, reliable under varying conditions, and capable of consistent performance. The motivation behind this project is to design a system that meets these real-world requirements by prioritizing robustness, simplicity, and scalability. Ultimately, the goal is to create a surveillance solution that not only improves efficiency and accuracy but also aligns with the evolving needs of modern embedded systems and smart security applications.

1.2 INNOVATIVE CONTENT IN THE PROJECT

The proposed project introduces several innovative features that enhance the efficiency and reliability of modern surveillance systems. A key innovation is the use of **multi-sensor fusion**, where different sensing technologies work together to improve detection accuracy. Unlike conventional systems that rely on a single sensor and are prone to false alarms, this system integrates **Infrared (IR)** proximity sensing, IR break-beam detection, and ultrasonic distance measurement. Each sensor provides a distinct mode of detection, allowing the system to validate intrusion events through multiple inputs, thereby significantly reducing false positives.

Another important innovation is the implementation of an **event-driven monitoring approach**. Instead of continuous recording, the system activates the camera only when a valid intrusion is detected. This reduces unnecessary power consumption P and data storage, making the system more efficient and suitable for long-term deployment, especially in **Internet of Things (IoT)**-based environments.

The system also features an **automated camera orientation mechanism**, where a servo motor adjusts the camera's angular position θ toward the detected region. This ensures that the captured images are relevant and focused, improving the overall effectiveness of surveillance.

In addition, the use of a **Finite State Machine (FSM)** in the software design enables structured and controlled system operation. By managing different states such as detection, capture, and reset, the FSM prevents redundant triggering and ensures stable system performance over time t .

Overall, the innovation of this project lies in the intelligent integration of sensing, decision-making, and response mechanisms, resulting in a smart, efficient, and scalable intrusion detection system.

1.3 BLOCK SCHEMATIC

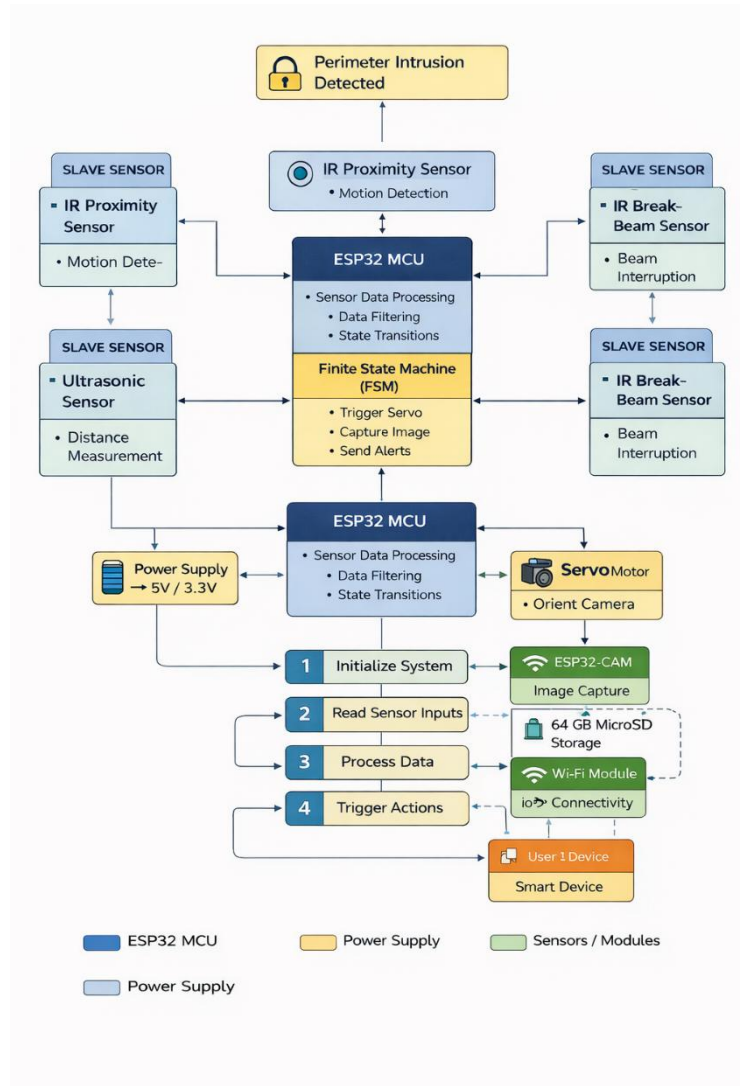


Fig: 1 Overall System schematic

1.3.1 Module Descriptions

1. Input Sensing Stage

- The system uses three primary sensors:
 - IR Proximity Sensor: Detects nearby motion based on reflected Infrared (IR) signals.
 - IR Break-Beam Sensor: Detects intrusion when the IR beam is interrupted, ensuring accurate boundary detection.
 - Ultrasonic Sensor: Measures distance *d* using time delay *t* of ultrasonic waves to detect approaching objects.

- These sensors continuously monitor the environment and generate signals when an object is detected.

2. Data Acquisition by ESP32

- All sensor outputs are connected to the ESP32 microcontroller (MCU) through GPIO pins.
- The ESP32 continuously reads sensor inputs in real time.
- Analog and digital signals from sensors are converted into usable data for processing.

3. Data Processing and Decision Making

- The ESP32 processes sensor inputs using embedded logic.
- A Finite State Machine (FSM) is implemented to manage system operation.
- The FSM divides the system into structured states such as:
 - Initialization
 - Detection
 - Action
 - Reset / Cooldown
- Sensor data is evaluated based on predefined thresholds and conditions to determine valid intrusion.

4. Software Execution Flow

- The system follows a sequential software workflow:
 1. Initialize System: Configure sensors, GPIO pins, and communication modules.
 2. Read Sensor Inputs: Continuously monitor IR and ultrasonic sensors.
 3. Process Data: Analyze sensor outputs and validate intrusion using logic/FSM.
 4. Trigger Actions: Execute required outputs when intrusion is confirmed.

5. Actuation and Camera Orientation

- When intrusion is detected:
 - The servo motor is activated.
 - It rotates to a specific angle θ using PWM signals.
- This aligns the camera toward the detected region for accurate monitoring.

6. Image Capture and Storage

- The ESP32 sends a trigger signal to the ESP32-CAM module.
- The camera captures images of the intrusion event.
- Captured images can be stored in a microSD card for future reference.

7. Communication and Livestreaming

- The ESP32-CAM connects to Wi-Fi using built-in capabilities.
- A web server is hosted to provide a livestream interface.
- Real-time video feed is transmitted to the user through a browser.

8. User Monitoring Interface

- The livestream can be accessed on a smartphone or PC.
- Users can monitor intrusion events in real time.
- This ensures immediate visual verification of detected activity.

9. Power Supply and Supporting Components

- The system is powered using a regulated +5V supply.
- The ESP32 operates at 3.3V logic level.
- Supporting components:
 - Capacitors for voltage stabilization
 - Resistors for signal conditioning
- These ensure reliable and noise-free operation.

10. Overall System Operation

- The system operates in an event-driven manner, activating only when intrusion is detected.
- Integration of sensing, processing, and communication ensures:
 - Reduced false alarms
 - Efficient power usage
 - Real-time monitoring capability

1.3.2 Cost of the Project

Total cost of the project comes up to Rs. 3000

S. No.	Component	Specification / Description	Quantity	Approx. Cost (₹)	Total (₹)
1	ESP32-CAM (AiThinker)	Wi-Fi + Camera module	1	600	600
2	ESP32 DevKit	Main controller board	1	350	350
3	IR Proximity Sensor	Digital output (3.3–5V)	1	100	100
4	IR Break-beam (SEN0503)	Transmitter–Receiver pair	1	150	150
5	Ultrasonic Sensor (HC-SR04)	Distance measurement	1	100	100
6	Servo Motor (MG90S, 180°)	Metal gear micro servo	1	200	200
7	Power Supply	USB power bank / 18650 setup	1	700	700
8	Wiring & Connectors	Jumper wires, pin headers	1 set	120	120
9	Decoupling Capacitor	470–1000 μ F	2	20	40
10	MicroSD Card (16 GB)	For camera image storage	1	250	250
11	Mounts / Enclosure	For fixing camera and board	1	250	250
12	FTDI / USB-TTL Module	For ESP32-CAM programming	1	150	150

TABLE-1: Cost Analysis

1.3.3 Advantages and Disadvantages

Advantages

- **Multi-sensor fusion:** Combines Infrared (IR) proximity, IR break-beam, and ultrasonic sensors, improving detection accuracy and significantly reducing false alarms.
- **Event-driven operation:** Camera is activated only during valid intrusion events, reducing power consumption P and unnecessary data storage.
- **Automated camera orientation:** Servo motor adjusts the camera angle θ toward the detected region, ensuring focused and relevant image capture.
- **High reliability:** Use of multiple sensing mechanisms ensures detection is validated through independent inputs, increasing system trustworthiness.
- **Finite State Machine (FSM) control:** Structured operation through states such as detection, capture, and reset prevents repeated triggering and ensures stable performance over time t .
- **Fast response time:** System reacts quickly ($\approx 500\text{--}700$ ms), enabling real-time intrusion detection and image capture.
- **Cost-effective solution:** Total system cost is approximately ₹3000, making it suitable for wide-scale deployment.
- **Scalability:** Modular design allows easy integration with additional sensors or future technologies such as Internet of Things (IoT).

Disadvantages

- **Limited detection range:** IR break-beam works only along a fixed path, and ultrasonic sensing is limited to short distances.
- **Environmental dependency:** IR sensors can be affected by lighting conditions, while ultrasonic readings may vary due to surface properties and noise.
- **No real-time remote monitoring:** Current system stores images locally without instant alerts or cloud integration.
- **Limited processing capability:** ESP32-CAM has constraints in handling advanced features like real-time video streaming or AI-based detection.
- **Power stability concerns:** Components such as servo motors and camera modules may introduce voltage fluctuations if not properly managed.

1.4 OVERVIEW OF THE REPORT

This report presents the design, development, and implementation of an ESP32-based multi-sensor intrusion detection and smart camera system. The document is organized into multiple chapters, each focusing on a specific aspect of the project to provide a clear and structured understanding of the system.

Chapter 1 introduces the project, outlining the motivation behind developing an energy-efficient and reliable surveillance system. It also highlights the innovative aspects of the design, explains the system architecture at a conceptual level, and discusses the advantages, limitations, and overall scope of the project.

Chapter 2 provides a literature survey on existing intrusion detection and surveillance systems. It reviews conventional approaches and recent advancements in multi-sensor detection, embedded systems, and smart monitoring solutions, establishing the need for the proposed system.

Chapter 3 focuses on the design and simulation of the system. It explains the overall system design, working principle, and software implementation using the ESP32 microcontroller. The chapter also describes the use of a Finite State Machine (FSM) for structured operation and discusses simulation tools used for circuit and system design.

Chapter 4 presents the hardware implementation of the system. It includes detailed descriptions of components such as sensors, ESP32, servo motor, and camera module. The chapter also explains the circuit schematic, system integration, and the results obtained from testing, including performance analysis and system response.

Finally, the report concludes with a summary of the project outcomes and discusses possible future enhancements, such as IoT integration, AI-based detection, and improved communication capabilities.

CHAPTER - 2

LITERATURE SURVEY

2.1 SURVEY ON IOT-BASED INTRUSION DETECTION SYSTEMS

Intrusion detection systems have undergone significant transformation with the emergence of Internet of Things (IoT) technology. Traditional surveillance systems primarily relied on standalone devices such as CCTV cameras and alarm systems, which lacked real-time responsiveness, automation, and remote accessibility. In contrast, IoT-based systems integrate sensors, embedded controllers, and communication modules to enable intelligent monitoring and automated decision-making [1]. These systems allow users to access real-time data through mobile or web-based interfaces, enhancing usability and control.

Afzaal et al. (2024) proposed an IoT-assisted monitoring system that combines sensing units, data acquisition modules, and wireless communication protocols to improve real-time responsiveness and scalability [2]. Their work demonstrates how distributed sensing and cloud connectivity can enhance system performance. Similarly, several studies emphasize the role of IoT platforms such as Blynk and Firebase in enabling remote monitoring and alert systems. However, a key limitation observed in IoT-based intrusion detection systems is their dependence on continuous network connectivity. In environments with limited or unstable internet access, system performance may degrade significantly. Additionally, security concerns such as data privacy and unauthorized access remain critical challenges in IoT-based implementations.

2.2 SURVEY ON ESP32-BASED SMART SURVEILLANCE SYSTEMS

The ESP32 microcontroller has emerged as a powerful platform for developing smart surveillance systems due to its integrated Wi-Fi, dual-core processing capability, and low cost. Its ability to handle both sensing and communication tasks makes it suitable for real-time embedded applications. Abordo et al. (2024) developed a smart surveillance system using the ESP32-CAM module combined with motion detection techniques [3]. Their system demonstrated efficient image capture and transmission, highlighting the potential of ESP32 in modern security applications.

Other research works have explored the use of ESP32 in home security systems integrated with mobile applications for real-time alerts [4]. These systems utilize Wi-Fi connectivity

to send notifications when motion is detected. While such implementations improve accessibility and automation, they often rely on single-sensor detection methods such as PIR or IR sensors. This makes them vulnerable to false alarms caused by environmental disturbances such as lighting changes or temperature fluctuations. Furthermore, the limited processing capability of ESP32 restricts the implementation of advanced features such as real-time video analytics without additional hardware support.

2.3 SURVEY ON MULTI-SENSOR INTRUSION DETECTION APPROACHES

To overcome the limitations of single-sensor systems, recent research has focused on multi-sensor integration techniques. Multi-sensor systems combine different sensing mechanisms to improve detection accuracy and reliability. A study on advanced surveillance systems integrated IR sensors, ultrasonic sensors, and camera modules to achieve enhanced monitoring and access control [5]. The results showed that combining multiple sensors significantly reduces false positives and improves system robustness.

Similarly, research on smart home security systems demonstrates that sensor fusion enables cross-verification of intrusion events, ensuring that detection is confirmed through multiple independent inputs [6]. This approach enhances reliability and reduces the chances of false triggering. Multi-sensor systems also provide flexibility in adapting to different environments, as each sensor compensates for the limitations of others. However, integrating multiple sensors increases system complexity and requires efficient data processing algorithms to manage inputs effectively.

2.4 SURVEY ON ULTRASONIC-BASED DETECTION SYSTEMS

Ultrasonic sensors are widely used in embedded systems for distance measurement and object detection. These sensors operate by transmitting ultrasonic waves and calculating the time delay t_{of} of the reflected signal to determine the distance d of an object [7]. Their ability to function independently of lighting conditions makes them suitable for both indoor and outdoor applications.

Research studies have demonstrated the effectiveness of ultrasonic sensors in security systems, particularly when combined with rotating mechanisms for wider area coverage [7]. These systems can detect approaching objects and trigger alarms or other responses. However, ultrasonic sensors are sensitive to environmental factors such as surface texture,

angle of incidence, and acoustic noise, which may affect measurement accuracy. Despite these limitations, they remain a reliable component when integrated with other sensing technologies.

2.5 SURVEY ON CAMERA-BASED INTRUSION DETECTION SYSTEMS

Camera-based surveillance systems are an essential component of modern security solutions, as they provide visual evidence of intrusion events. The ESP32-CAM module has been widely used in such systems due to its compact size, low cost, and wireless capabilities. Research by Abordo et al. (2024) demonstrated that ESP32-CAM-based systems can capture and transmit images in real time, making them suitable for smart monitoring applications [3].

Further studies indicate that integrating cameras with sensor-based triggering mechanisms significantly improves system efficiency. Instead of continuous recording, cameras can be activated only when an intrusion is detected, reducing power consumption and storage requirements [5]. However, camera-based systems face challenges such as limited processing power, memory constraints, and dependency on proper lighting conditions. Advanced applications such as real-time object detection require additional computational resources, which may not be feasible in low-cost embedded systems.

2.6 SURVEY ON ACTUATION AND CONTROL MECHANISMS

Actuation systems, particularly servo motors, play a crucial role in enhancing the functionality of surveillance systems. Servo motors enable precise angular positioning and are widely used in applications such as robotics, automation, and camera control. Studies show that integrating servo motors in surveillance systems allows dynamic adjustment of camera orientation, improving coverage and monitoring efficiency.

Control mechanisms such as Finite State Machines (FSM) are commonly used in embedded systems to ensure structured and reliable operation. FSM-based designs divide system operation into distinct states, enabling efficient handling of multiple events without conflicts. Research indicates that FSM implementation improves system stability, reduces redundant operations, and enhances real-time performance in embedded applications.

2.7 RESEARCH GAP AND MOTIVATION

From the reviewed literature, it is evident that significant advancements have been made in intrusion detection systems; however, several limitations still persist. Many existing systems rely on single-sensor detection, leading to false alarms and reduced reliability. Continuous monitoring approaches result in high power consumption and excessive data storage, making them inefficient for long-term deployment. Additionally, IoT-based systems often depend heavily on network connectivity, which may limit their usability in standalone environments.

Furthermore, while camera-based systems provide visual verification, they often lack intelligent triggering mechanisms, resulting in inefficient operation. Similarly, systems that incorporate multiple sensors may face challenges in integrating and processing data effectively.

Therefore, there is a clear need for a system that combines multi-sensor fusion, event-driven operation, and intelligent control mechanisms. The proposed system addresses these gaps by integrating multiple sensing technologies with automated camera orientation and efficient processing using the ESP32 microcontroller. This results in a reliable, energy-efficient, and scalable intrusion detection solution suitable for modern surveillance applications.

CHAPTER - 3

DESIGN AND SIMULATION

3.1 DESIGN OF THE PROJECT

3.1.1 SYSTEM DESIGN ARCHITECTURE

The proposed system is designed using a modular architecture that integrates sensing, processing, and actuation units to achieve efficient intrusion detection and response. At the core of the system is the ESP32, a 32-bit microcontroller that serves as the central control unit. It interfaces with input sensors and output devices through its General-Purpose Input Output (GPIO) pins, enabling real-time data acquisition and system control.

The sensing unit consists of multiple sensors, including Infrared (IR) proximity sensors, IR break-beam sensors, and an ultrasonic sensor (HC-SR04). Each sensor provides distinct input data: IR proximity sensors detect motion based on changes in infrared radiation, IR break-beam sensors identify interruptions in a continuous beam for precise detection, and the ultrasonic sensor measures object distance by calculating the time delay of reflected ultrasonic waves. The combination of these sensors enhances detection accuracy through sensor fusion.

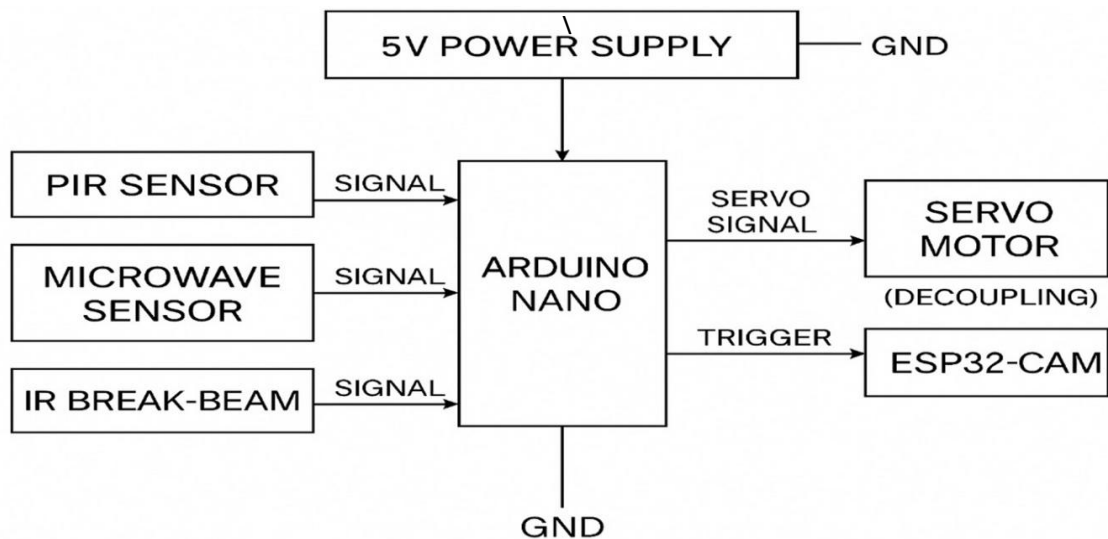


Fig: 2 Hardware Block Diagram

The processing unit, i.e., the ESP32, continuously monitors sensor inputs and executes decision-making logic based on predefined conditions. It analyses the incoming data to

determine the presence of an intrusion and identifies the corresponding detection region. Based on this evaluation, appropriate control signals are generated.

The output section includes a servo motor and an ESP32-CAM module. The servo motor is controlled using Pulse Width Modulation (PWM) signals and rotates to a specific angle depending on the detected region. The ESP32-CAM is triggered to capture images of the intrusion event, which are then stored locally on a Secure Digital (SD) card for verification.

Additionally, a regulated power supply unit provides both 5V and 3.3V outputs to different components. Supporting elements such as resistors and capacitors are used for voltage stabilization and noise reduction, ensuring reliable system performance. Overall, the design emphasizes modularity, efficiency, and scalability, making it suitable for practical deployment.

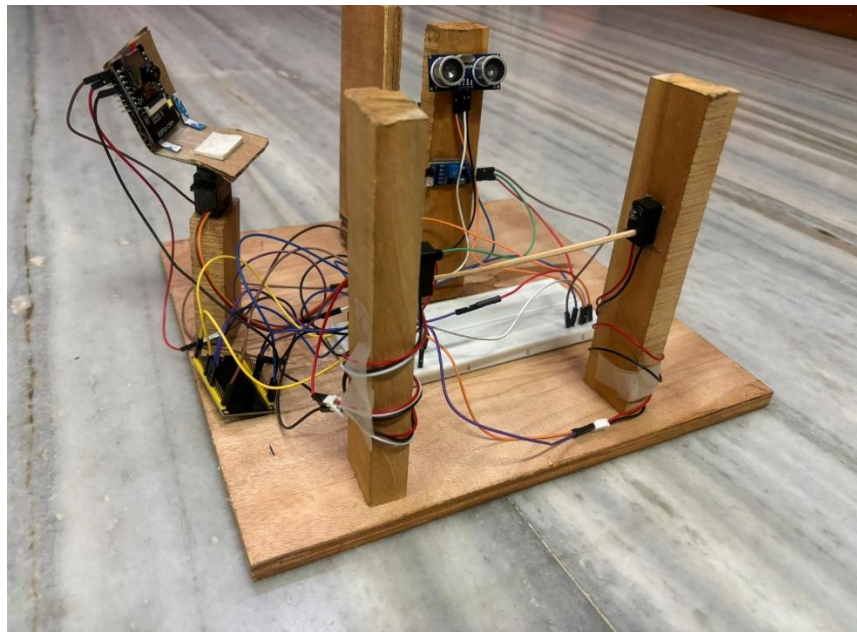


Fig: 3 Complete Hardware Setup

3.1.2 WORKING PRINCIPLE OF THE SYSTEM

The working principle of the system is based on continuous monitoring of environmental conditions using multiple sensors and executing appropriate actions upon detecting an intrusion. The system operates in a real-time loop, where sensor inputs are continuously read and analysed by the ESP32 controller.

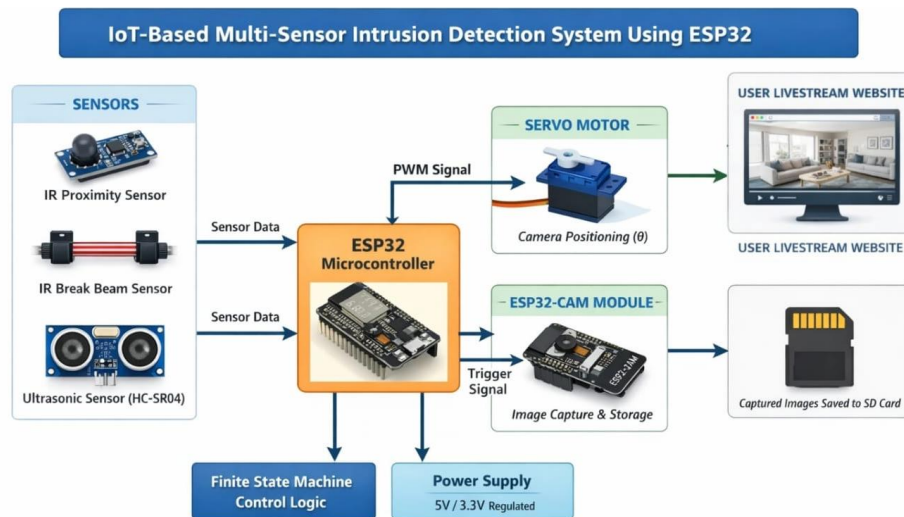


Fig: 4 System Principle

The IR proximity sensor detects motion by sensing variations in infrared radiation emitted by objects or human presence. The IR break-beam sensor operates by transmitting a continuous infrared beam between a transmitter and receiver; when this beam is interrupted, it indicates a boundary breach. The ultrasonic sensor measures the distance d of objects by emitting ultrasonic pulses and calculating the time delay t of the reflected signal. These three sensing mechanisms work together to ensure accurate and reliable detection.

When any of the sensors detect an intrusion, the ESP32 processes the input signals and determines the type and location of the event. Based on this information, the controller generates a PWM signal to actuate the servo motor. The servo motor then rotates to a predefined angle θ , aligning the camera toward the detected region.

Simultaneously, the ESP32 sends a trigger signal to the ESP32-CAM module, initiating image capture. The captured image is stored in the SD card, allowing later access for

verification and analysis. An LED indicator is also activated during detection, providing a visual indication of system activity.

After completing the capture process, the servo motor returns to its initial position, and the system resumes its monitoring state. This event-driven operation ensures efficient use of resources by avoiding unnecessary processing and recording. The integration of multiple sensors and automated response mechanisms improves detection accuracy and system reliability.

3.1.3 SOFTWARE DESIGN AND CONTROL STRATEGY

The software design is implemented using the Arduino Integrated Development Environment (IDE) and follows a Finite State Machine (FSM) approach. This method organizes system operation into well-defined states, ensuring structured and conflict-free execution of tasks.

The FSM consists of states such as READY, DETECTION, TARGET SETTLE, CAPTURE, RETURN, and COOLDOWN. In the READY state, the system continuously monitors sensor inputs through GPIO pins. When an intrusion is detected, it transitions to the DETECTION state, where the triggered sensor is identified and processed.

In the TARGET SETTLE state, the servo motor is positioned to a specific angle θ based on the detected event. Once stabilized, the system moves to the CAPTURE state, where a signal is sent to the ESP32-CAM module to capture an image. After capturing, the system enters the RETURN state, bringing the servo back to its default position.

The COOLDOWN state prevents repeated triggers within a short time interval t , reducing redundancy and improving system stability. The FSM approach ensures efficient timing, minimizes processing conflicts, and supports real-time performance.

The software also includes functions for sensor reading, servo control, and camera triggering. PWM is used for accurate servo positioning, while interrupt-based triggering improves camera efficiency. Overall, the design ensures smooth coordination between sensing, processing, and actuation units.

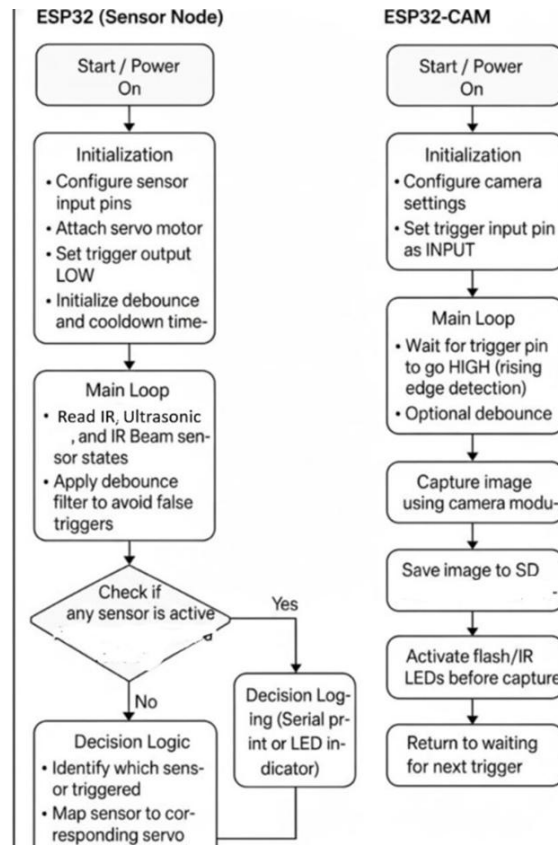


Fig: 5 Software Architecture

3.2 SIMULATION

The simulation of the proposed system is carried out through embedded programming using the Arduino Integrated Development Environment (IDE). The code implements the complete functionality of the intrusion detection system by integrating sensor inputs, decision-making logic, actuation control, and camera-based image streaming.

The system is divided into two major modules:

1. ESP32 DevKit Control Logic
2. ESP32-CAM Livestream Module

3.2.1 ESP32 CONTROL LOGIC SIMULATION

The ESP32 DevKit code is responsible for real-time monitoring of sensor inputs and controlling the overall system operation. Multiple sensors including Infrared (IR) proximity sensor, IR break-beam sensor, and ultrasonic sensor are interfaced with the ESP32 through GPIO pins .

The ultrasonic sensor calculates the distance d of an object using the time delay t of reflected ultrasonic waves. Based on predefined threshold conditions, the system determines whether an intrusion has occurred.

The control logic follows a Finite State Machine (FSM) approach, which divides the system operation into different states such as:

- READY
- TARGET SETTLE
- CAPTURE HOLD
- RETURN SETTLE
- COOLDOWN

In the READY state, the system continuously monitors all sensor inputs. When any sensor detects intrusion, the ESP32 identifies the source of detection and assigns a corresponding servo angle θ . The servo motor is then actuated using Pulse Width Modulation (PWM) signals to orient the camera toward the detected region. Once the servo reaches the desired position, the system triggers the camera by sending a signal through a dedicated GPIO pin. After image capture, the servo returns to its initial position, and the system enters a cooldown period to prevent repeated triggering. This structured FSM-based implementation ensures efficient and conflict-free operation of the system.

3.2.2 ESP32-CAM LIVESTREAM SIMULATION

The ESP32-CAM module is programmed to provide real-time image streaming through a web interface. The code initializes the camera using appropriate pin configurations and sets parameters such as frame size and image quality for stable operation.

The module connects to a Wi-Fi network using predefined credentials and hosts a web server. Once connected, the system generates a local IP address, which can be accessed through a browser to view the live video stream. A key feature of the implementation is the use of an interrupt-based trigger mechanism. When the ESP32 DevKit detects an intrusion, it sends a trigger signal to the ESP32-CAM. This activates a visual flash using an LED and highlights the intrusion event during streaming. The livestream is implemented using an HTTP multipart stream, where consecutive image frames are continuously transmitted to the client browser. This creates a real-time video feed of the

monitored area. The delay between frames is optimized to ensure smooth and stable streaming performance.

3.2.3 INTEGRATED SYSTEM BEHAVIOUR

The integration of both modules results in a coordinated system where detection, actuation, and visualization occur seamlessly. The ESP32 DevKit performs real-time sensing and decision-making, while the ESP32-CAM provides visual confirmation through a livestream website.

When an intrusion is detected:

- Sensors generate input signals
- ESP32 processes inputs and determines detection type
- Servo motor aligns camera to angle θ
- Camera is triggered and livestream highlights the event
- System resets and continues monitoring

This simulation demonstrates the effectiveness of combining multi-sensor detection with real-time image streaming for enhanced surveillance performance.

3.2.4 CODE

1. DEV-KIT

```
// ESP32 DevKit Controller Code
#include <Arduino.h>
#define IR_PROX_PIN 33
#define IR_BEAM_PIN 26
#define US_TRIG_PIN 5
#define US_ECHO_PIN 18
#define LED_PIN 25
#define SERVO_PIN 14
#define CAM_TRIGGER_PIN 23
#define BEAM_INTACT_LEVEL HIGH
#define PROX_ACTIVE_LEVEL LOW // ← FIXED
#define US_DETECT_CM 10.0f
// Servo PWM
#define SERVO_FREQ_HZ 50
#define SERVO_RES_BITS 16
#define SERVO_MIN_US 500
#define SERVO_MAX_US 2500

int servoHome = 120;
int servoProxAngle = 150;
```

```

int servoBeamAngle = 90;
int servoUSAngle = 150;

#define SERVO_SETTLE_MS 350
#define CAPTURE_HOLD_MS 1500

#define RETURN_SETTLE_MS 350
#define COOLDOWN_MS 600

enum SystemState { READY, TARGET_SETTLE, CAPTURE_HOLD,
RETURN_SETTLE, COOLDOWN };
SystemState sysState = READY
unsigned long stateStart = 0;
int targetAngle = 90;

// ----- Ultrasonic -----
float readUltrasonicCM() {
    digitalWrite(US_TRIG_PIN, LOW);
    delayMicroseconds(2);

    digitalWrite(US_TRIG_PIN, HIGH);
    delayMicroseconds(10);

    digitalWrite(US_TRIG_PIN, LOW);

    unsigned long duration = pulseIn(US_ECHO_PIN, HIGH,
25000UL);
    if (duration == 0) return -1.0;

    return (duration * 0.0343) / 2.0;
}

// ----- Servo -----
void setServoAngle(int angle) {
    angle = constrain(angle, 0, 180);

    int us = SERVO_MIN_US +
        ((SERVO_MAX_US - SERVO_MIN_US) * angle) / 180;

    uint32_t duty = ((1 << SERVO_RES_BITS) - 1) * us / 20000;

    ledcWrite(SERVO_PIN, duty);
}

// ----- Camera Trigger -----
void triggerCamera() {
    Serial.println("📷 Triggering Camera");

    digitalWrite(CAM_TRIGGER_PIN, LOW);

```

```

    delay(CAPTURE_HOLD_MS);
    digitalWrite(CAM_TRIGGER_PIN, HIGH);
}

// ----- Setup -----
void setup() {
    Serial.begin(115200);
    Serial.println("\nIntrusion Sentinel - IR Prox + Beam +
Ultrasonic");

    pinMode(IR_PROX_PIN, INPUT);
    pinMode(IR_BEAM_PIN, INPUT);
    pinMode(US_TRIG_PIN, OUTPUT);
    pinMode(US_ECHO_PIN, INPUT);
    pinMode(LED_PIN, OUTPUT);

    pinMode(CAM_TRIGGER_PIN, OUTPUT_OPEN_DRAIN);
    digitalWrite(CAM_TRIGGER_PIN, HIGH);

    ledcAttach(SERVO_PIN, SERVO_FREQ_HZ, SERVO_RES_BITS);

    setServoAngle(servoHome);
    digitalWrite(LED_PIN, LOW);

    Serial.println("System ready!");
}

// ----- Main Loop -----
void loop() {

    bool    proxDetect    =    (digitalRead(IR_PROX_PIN)    ==
PROX_ACTIVE_LEVEL);
    bool    beamBroken    =    (digitalRead(IR_BEAM_PIN)    !=
BEAM_INTACT_LEVEL);

    float dist1 = readUltrasonicCM();
    delay(8);
    float dist2 = readUltrasonicCM();

    float distance = (dist1 < 0 && dist2 < 0) ? -1.0 :
min(dist1, dist2);
    bool usDetect = (distance > 0 && distance <= US_DETECT_CM);

    bool anyDetect = proxDetect || beamBroken || usDetect;

    digitalWrite(LED_PIN, anyDetect ? HIGH : LOW);

    switch (sysState) {

```

```

case READY:
    if (anyDetect) {

        if (beamBroken)
            targetAngle = servoBeamAngle;
        else if (proxDetect)
            targetAngle = servoProxAngle;
        else if (usDetect)
            targetAngle = servoUSAngle;

        setServoAngle(targetAngle);

        sysState = TARGET_SETTLE;
        stateStart = millis();

        Serial.printf("Detection -> Servo %d° | prox=%d
beam=%d us=%d\n",
                    targetAngle, proxDetect, beamBroken,
usDetect);
    }
    break;

case TARGET_SETTLE:
    if (millis() - stateStart >= SERVO_SETTLE_MS) {

        triggerCamera();
        setServoAngle(servoHome);

        sysState = RETURN_SETTLE;
        stateStart = millis();

        Serial.println("Capture done. Returning home...");
    }
    break;

case RETURN_SETTLE:
    if (millis() - stateStart >= RETURN_SETTLE_MS) {
        sysState = COOLDOWN;
        stateStart = millis();
    }
    break;

case COOLDOWN:
    if (millis() - stateStart >= COOLDOWN_MS) {
        sysState = READY;
        Serial.println("System ready for next detection.");
    }
    break;
}

```

```
    delay(5);  
}
```

2. ESP-32 CAM

```
#include "esp_camera.h"  
#include <WiFi.h>  
#include <WebServer.h>  
#include "soc/soc.h"  
#include "soc/rtc_cntl_reg.h"  
  
#define CAMERA_MODEL_AI_THINKER  
#include "camera_pins.h"  
  
const char* ssid = "gop";  
const char* password = "pika1111";  
  
WebServer server(80);  
// Trigger from DevKit  
#define TRIGGER_PIN 13  
// Flash LED  
#define FLASH_LED 4  
volatile bool triggerFlag = false;  
  
// ===== INTERRUPT =====  
void IRAM_ATTR triggerISR() {  
    triggerFlag = true;  
}  
  
// ===== STREAM =====  
void handleStream() {  
    WiFiClient client = server.client();  
  
    String header = "HTTP/1.1 200 OK\r\n";  
    header += "Content-Type: multipart/x-mixed-replace;  
boundary=frame\r\n\r\n";  
    server.setContent(header);  
  
    while (client.connected()) {  
        camera_fb_t * fb = esp_camera_fb_get();  
        if (!fb) break;  
        if (triggerFlag) {  
            digitalWrite(FLASH_LED, HIGH);  
            delay(300); // visible flash  
            digitalWrite(FLASH_LED, LOW);  
            triggerFlag = false;  
        }  
    }  
}
```



```

    }
    client.print("--frame\r\n");
    client.print("Content-Type: image/jpeg\r\n\r\n");
    client.write(fb->buf, fb->len);
    client.print("\r\n");
    esp_camera_fb_return(fb);
    delay(150); // stable stream
}
}

// ===== ROOT =====
void handleRoot() {
    String html = "<html><body>";
    html += "<h2>ESP32-CAM LIVE STREAM</h2>";
    html += "<p>Camera Ready</p>";
    html += "<img src='/stream'>";
    html += "</body></html>";

    server.send(200, "text/html", html);
}

// ===== SETUP =====
void setup() {
    Serial.begin(115200);
    // Disable brownout
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

    // LED setup
    pinMode(FLASH_LED, OUTPUT);
    digitalWrite(FLASH_LED, LOW);
    // Trigger pin setup
    pinMode(TRIGGER_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(TRIGGER_PIN), triggerISR,
FALLING);
    // CAMERA CONFIG
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;

    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;

    config.pin_xclk = XCLK_GPIO_NUM;

```

```

config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;

config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;

config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;

config.xclk_freq_hz = 100000000;
config.pixel_format = PIXFORMAT_JPEG;

config.frame_size = FRAMESIZE_QQVGA;
config.jpeg_quality = 30;
config.fb_count = 1;
config.fb_location = CAMERA_FB_IN_DRAM;

if (esp_camera_init(&config) != ESP_OK) {
    Serial.println("Camera init failed");
    return;
}

Serial.println("Camera OK");

// ===== WIFI =====
WiFi.begin(ssid, password);
Serial.print("Connecting");

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("\nConnected!");
Serial.print("Open this URL: http://");
Serial.println(WiFi.localIP());

// ===== SERVER =====
server.on("/", handleRoot);
server.on("/stream", handleStream);

server.begin();
}

// ===== LOOP =====
void loop() {
    server.handleClient();
}

```

3.3 SCREENSHOTS OF THE SIMULATED RESULTS

```
PROX=0 BEAM=0 US=1
Ultrasonic Distance: 4.25 cm
Servo Angle: 150
Camera Triggered
-----
Sensor Status:
PROX=1 BEAM=0 US=0
Servo Angle: 150
Camera Triggered
-----
Sensor Status:
PROX=0 BEAM=1 US=0
Servo Angle: 90
```

Fig: 6 IDE Environment Results

CHAPTER - 4

HARDWARE

4.1 HARDWARE DESCRIPTION

4.1.1 SENSING AND PROCESSING UNIT

- **ESP32 DevKit (Controller):**

The ESP32 acts as the central processing unit of the system, coordinating all operations. It continuously reads inputs from the sensors through GPIO pins, processes the data using embedded logic, and generates appropriate control signals for the output devices. Its built-in Wi-Fi capability also enables communication with the ESP32-CAM for livestream functionality.

- **IR Proximity Sensor:**

This sensor detects nearby objects by measuring reflected Infrared (IR) radiation. When an object comes within its range, the reflected IR signal changes, indicating motion. It is mainly used for short-range detection and provides fast response to movement.

- **IR Break-Beam Sensor:**

The break-beam sensor consists of an IR transmitter and receiver placed opposite each other. A continuous IR beam is maintained between them, and when an object interrupts this beam, the receiver detects a signal change. This allows precise line-of-sight detection, making it suitable for boundary monitoring.

- **Ultrasonic Sensor (HC-SR04):**

The ultrasonic sensor measures the distance d of objects by transmitting ultrasonic waves and calculating the time delay t of the reflected signal. It helps detect approaching objects within a predefined threshold range and works effectively regardless of lighting conditions.

4.1.2 ACTUATION, IMAGING AND SUPPORTING UNIT

- **Servo Motor:**

The servo motor is used to orient the camera toward the detected region. It rotates to a specific angle θ based on the sensor input and is controlled using Pulse Width Modulation (PWM) signals from the ESP32, allowing precise and smooth positioning.

- **ESP32-CAM Module:**

This module is responsible for capturing images and providing a livestream through a web interface. When triggered by the ESP32, it captures the intrusion event and allows real-time monitoring via a browser, enhancing system usability.

- **LED Indicator:**

The LED serves as a visual status indicator. It turns ON when an intrusion is detected or when the system is active, helping in quick identification of system operation.

- **Power Supply Unit:**

The system operates on a regulated +5V supply, which is distributed to all components. The ESP32 and logic circuits operate at 3.3V, ensuring compatibility and safe functioning of the system.

- **Voltage Divider (Ultrasonic Interface):**

A voltage divider circuit using resistors (10kΩ and 20kΩ) is used to step down the 5V ECHO signal of the ultrasonic sensor to 3.3V. This protects the ESP32 input pins from overvoltage and ensures reliable signal reading.

- **Decoupling Capacitors:**

Capacitors such as 100μF and 100nF are placed across power lines to filter noise and stabilize voltage levels. They are especially important during sudden current changes caused by the servo motor and camera module.

4.1.3 HARDWARE SPECIFICATIONS

S. No.	Component	Technical Specifications
1	ESP32 DevKit V1	Dual-core 32-bit Tensilica LX6 MCU; 2.4 GHz Wi-Fi + Bluetooth; Operating voltage: 3.3 V; Input voltage: 5 V (via USB or VIN); GPIO: 30 pins
2	ESP32-CAM Module	OV2640 camera; 2 MP resolution; SD card support up to 4 GB; Power supply: 5 V; Interface: UART / GPIO

3	IR Proximity Sensor (HC-SR501)	Operating voltage: 4.5–20 V; Output: 3.3 V logic HIGH on detection;
4	IR Break-Beam Sensor (SEN0503)	Supply voltage: 3.3–5 V; Output: digital HIGH/LOW; Range: 2–30 cm; Output type: Active LOW
5	Ultrasonic Sensor (HC-SR04)	Operating voltage: 5 V; Range: 2–400 cm; Frequency: 40 kHz; Output: TTL 5 V; Interface: TRIG, ECHO pins
6	Servo Motor (SG90)	Operating voltage: 4.8–6 V; Rotation: 0°–180°; Torque: 1.8 kg·cm; PWM control signal
7	LED Indicator (5 mm)	Forward voltage: 2–2.2 V; Forward current: 20 mA; Color: Red
8	Power Supply (USB / DC Adapter)	Input: 5 V DC; Current capacity: ≥ 2 A

Table -2 : Hardware specifications

4.2 RESULTS AND DISCUSSION

- The ESP32-based multi-sensor intrusion detection system was successfully implemented and tested under various conditions to evaluate performance, accuracy, and reliability.
- The system effectively detected intrusion events using **Infrared (IR) proximity, IR break-beam, and ultrasonic sensors**, ensuring accurate and real-time monitoring.

Sensor performance:

- IR proximity sensor provided quick response to nearby motion.
- IR break-beam sensor ensured precise detection when the beam path was interrupted.
- Ultrasonic sensor measured distance *d* using time delay *t* and detected approaching objects within the threshold range.
- The combination of multiple sensors enabled **sensor fusion**, which significantly reduced false alarms compared to single-sensor systems.

- The system achieved a **fast response time** ($\approx 500\text{--}700$ ms), including detection, processing, servo movement, and camera triggering.
- The **servo motor** successfully rotated to the required angle θ , aligning the camera toward the detected intrusion region.
- The **ESP32-CAM module** captured images effectively and provided a stable livestream through a web interface, accessible via a local IP address.
- The implementation of a **Finite State Machine (FSM)** ensured structured system operation by managing states such as detection, capture, return, and cooldown, preventing repeated triggering.

Implementation challenges:

- Integration of the camera module required careful synchronization of trigger signals and timing.
- Manual calibration of servo angles θ was required to align the camera accurately with different detection zones.
- Multiple iterations were needed to achieve proper positioning and reliable image capture.

Observed limitations:

- IR sensors were affected by environmental conditions such as ambient light and obstacles.
- Ultrasonic sensor accuracy varied with surface properties and noise.
- Minor voltage fluctuations occurred during simultaneous operation of servo motor and camera.
- Overall, the system demonstrated **reliable performance**, achieving efficient intrusion detection with reduced false alarms, automated camera positioning, and real-time livestream monitoring.

4.3 SCREENSHOTS OF THE HARDWARE USED AND RESULTS



Fig: 9 Detection of IR Proximity and Ultrasonic Sensors

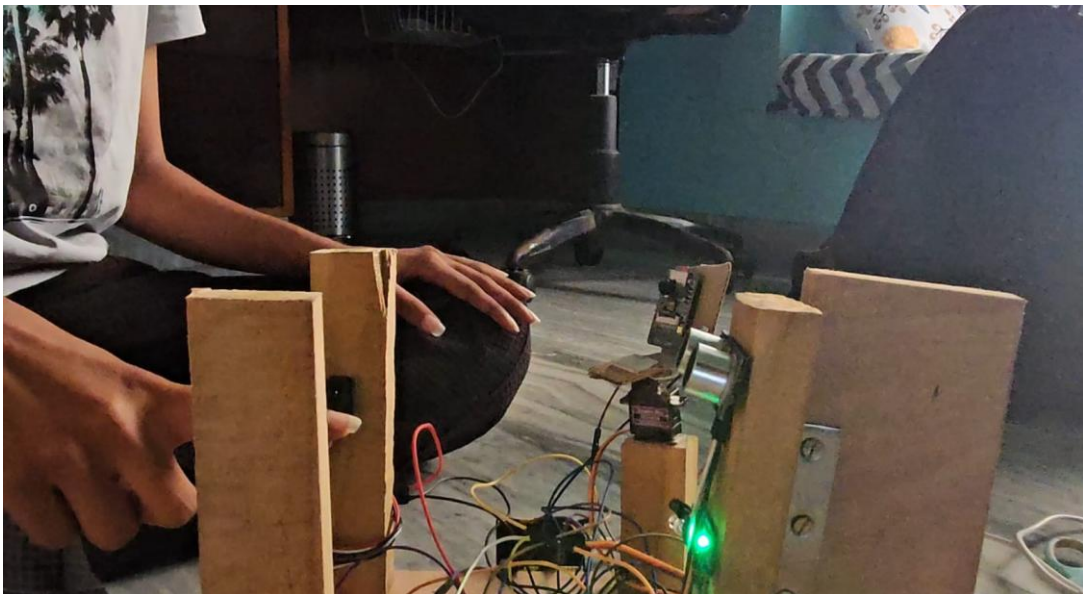


Fig: 10 Detection of IR Break Beam Sensor

CHAPTER -5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

The proposed ESP32-based multi-sensor intrusion detection system was successfully designed, developed, and implemented to address the limitations of conventional surveillance systems. By integrating multiple sensing mechanisms, including Infrared (IR) proximity, IR break-beam, and ultrasonic sensors, the system achieves accurate and reliable detection of intrusion events. The use of multi-sensor fusion ensures that detection is validated through multiple independent inputs, significantly reducing false alarms and improving overall system dependability.

The ESP32 microcontroller effectively performs real-time processing of sensor data and coordinates the operation of all system components. The implementation of a Finite State Machine (FSM) provides a structured control mechanism, enabling smooth transitions between detection, camera triggering, servo positioning, and reset states. This enhances system stability and prevents redundant or conflicting operations.

The integration of a servo motor for automated camera positioning further improves the system's effectiveness by ensuring that the camera is aligned toward the detected region. The ESP32-CAM module successfully captures images and provides a real-time livestream through a web interface, allowing users to monitor intrusion events efficiently. The event-driven operation of the system minimizes unnecessary power consumption and data storage, making it suitable for long-term deployment.

During implementation, practical challenges such as camera integration and manual calibration of servo angles were encountered, highlighting important considerations in real-world system design. Despite these challenges, the system demonstrated a fast response time, stable performance, and reliable operation under different testing conditions.

Overall, the project successfully meets its objectives of developing a cost-effective, efficient, and intelligent intrusion detection system. The combination of multi-sensor detection, automated response, and real-time monitoring makes the system a scalable and practical solution for modern security applications.

5.2 FUTURE SCOPE

- **IoT Integration:** The system can be enhanced by integrating cloud-based platforms to enable remote monitoring and instant alerts through mobile or web applications.
- **AI-Based Detection:** Incorporating machine learning or computer vision techniques can improve object recognition and enable intelligent classification of intruders.
- **Wireless Communication Expansion:** Technologies such as GSM, LoRa, or MQTT can be used for long-range communication and improved connectivity.
- **Automatic Calibration:** Servo motor positioning can be automated using feedback systems to eliminate manual angle calibration and improve accuracy.
- **Power Optimization:** Implementation of low-power modes and battery or solar-based operation can make the system more energy-efficient.
- **Enhanced Security Features:** Integration of alarm systems, facial recognition, or access control mechanisms can further improve system functionality.

CHAPTER -6

BIBLIOGRAPHY

- [1] Sharma, H., & Kanwal, N. (2024). Smart surveillance using IoT: A review. *Radioelectronic and Computer Systems*.
<https://nti.khai.edu/ojs/index.php/reks/article/view/reks.2024.1.10>
- [2] Afreen, H., et al. (2023). IoT-based smart surveillance system for high-security areas. *Applied Sciences*, 13(15), 8936. <https://www.mdpi.com/2076-3417/13/15/8936>
- [3] Hussain, A., et al. (2025). Smart surveillance: A review of recent advancements, applications, and ethical considerations. *ResearchGate*.
<https://www.researchgate.net/publication/391450427>
- [4] Sharma, H., et al. (2024). Smart surveillance using IoT in smart cities. *Scribd*.
<https://www.scribd.com/document/921528720>
- [5] Review on smart surveillance robot based on IoT. (2024). *International Journal of Computer Science Publications (IJCSP)*.
<https://rjpn.org/ijcspub/papers/IJCSP24B1044.pdf>
- [6] Arshad, J., et al. (2018). A review of performance, energy and privacy of intrusion detection systems for IoT. *arXiv*. <https://arxiv.org/abs/1812.09160>
- [7] Abdulmalek, S., et al. (2022). IoT-based monitoring systems: A review. *PMC*.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC9601552/>
- [8] Arduino. (n.d.). *Arduino IDE documentation*. <https://www.arduino.cc/en/software>
- [9] Espressif Systems. (2023). *ESP32 technical reference manual*.
<https://www.espressif.com/en/support/documents/technical-documents>
- [10] Random Nerd Tutorials. (n.d.). *ESP32-CAM video streaming and face recognition using Arduino IDE*. <https://randomnerdtutorials.com/esp32-cam-video-streaming-face-recognition-arduino-ide/>

